

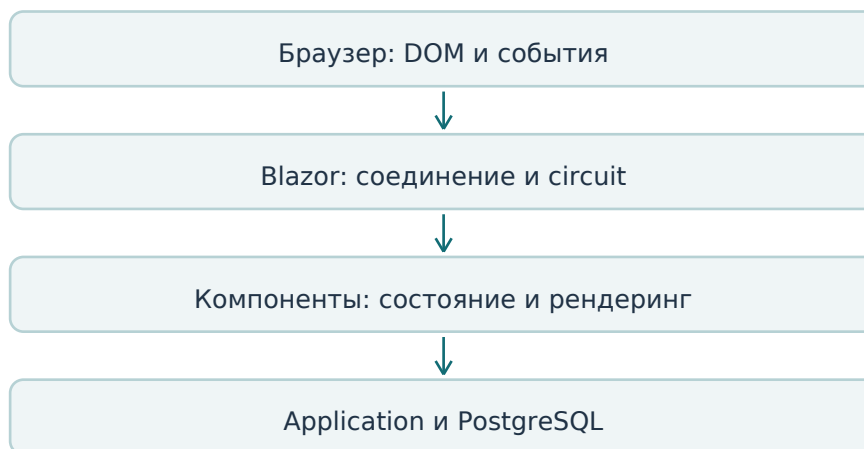
МЕХАНИЗМЫ ВМЕСТО ЗАКЛИНАНИЙ

Blazor Server

Как это работает

От HTTP-запроса и HTML до серверных компонентов, circuit, cookie-аутентификации и общения пользователей.

Последовательный учебник на примере Real-Time Chat. Для разработчика, который уже писал код, но хочет собрать цельную модель его выполнения.



.NET 10 • 40 страниц • объяснения, схемы и практика

Подготовлено для review проекта Real-Time Chat

Содержание

Как читать учебник	3
01-04. Браузер, HTTP, HTML и события	4
05-08. ASP.NET Core, DI и компоненты	8
09-12. SSR, режимы, запуск и circuit	12
13-16. Клик, рендеринг и lifecycle	16
17-21. async, scopes, EF Core и уведомления	20
22-28. Cookie, вход, выход и навигация	25
29-32. Архитектура, выбор модели и deploy	32
Практика 1: три режима одной кнопки	36
Практика 2: вход и доставка сообщения	37
Опорные ответы для самопроверки	38
Источники и дальнейшее чтение	39

Маршрут для нашего текущего затруднения: разделы 02-04, затем 09-13 и 22-26. Полный маршрут: читать последовательно и проверять объяснения экспериментами.

Как читать этот учебник

Сначала восстановить механизм. Затем разобрать синтаксис.

Этот текст написан для нашего Real-Time Chat на .NET 10. Ты уже видел PHP, polling, WebSocket, SignalR и Blazor WASM. Теперь задача - связать знакомые части: кто исполняет код, где хранится состояние и как действие проходит от пользователя до базы и обратно. Знание названия метода ещё не отвечает на эти вопросы.

Главный сквозной пример: один серверный процесс, компоненты Blazor Server и ChatService, который обращается к репозиториям PostgreSQL. Здесь нет HTTP API между компонентом и Application. HTTP остаётся между браузером и сервером, а интерактивность использует отдельный механизм доставки событий.

Читай по 3-5 разделов за подход. После блока закрой текст и нарисуй путь одного действия. На следующий день восстанови тот же путь снова. Если воспроизвести не получилось, возвращайся к одному пробелу, а не перечитывай всё подряд. Цель - объяснить поведение, а не запомнить листинг.

Что означают примеры

Короткие фрагменты иллюстрируют механизм. Если явно не сказано обратное, они не являются полным готовым приложением: обработка ошибок, отмена, права доступа и очистка ресурсов могут быть разобраны на других страницах. Не вставляй все фрагменты подряд в проект.

Схемы показывают логические шаги. Они не обещают отдельный поток для каждой стрелки или один сетевой пакет на каждый шаг. Реальные протоколы группируют сообщения, а runtime распределяет работу между потоками.

Объяснения и примеры авторские. Ссылки на документацию приведены для проверки механизмов и дальнейшего чтения. Текст рассчитан на Blazor Web App в .NET 10; в старых проектах Blazor Server запуск и структура файлов отличаются.

Вернуться по памяти: Перед чтением: где, по твоему текущему представлению, находится экземпляр Room.razor? Что остаётся в браузере? Запиши ответы, даже если пока не уверен.

01. Две стороны одного приложения

Слово «страница» скрывает несколько разных объектов.

Когда ты открываешь чат, браузер и сервер не превращаются в один компьютер. У браузера есть DOM, поля ввода, CSS, JavaScript и сетевые соединения. У сервера есть процесс .NET, объекты компонентов, сервисы, подключения к базе. Нажатие мыши возникает в браузере. Метод `SendAsync` выполняется там, где размещён соответствующий компонент.

В WASM компонент работает в браузере. Чтобы сохранить сообщение, он пересекает сетевую границу: вызывает API или метод `Hub`. В `Interactive Server` компонент работает на сервере. Он может вызвать `ChatService` обычным C#-вызовом, но событие клика всё равно должно приехать из браузера. Сеть исчезает между компонентом и сервисом, а не между пользователем и приложением.

Браузер	Сервер
DOM, CSS, ввод с клавиатуры	Компоненты <code>Interactive Server</code>
Скрипт <code>Blazor</code> , обработка DOM	<code>ChatService</code> , репозитории, <code>Identity</code>
Полученные данные и локальное состояние	Память процесса и доступ к <code>PostgreSQL</code>

Представь две вкладки у разных людей. У каждой свой документ и своё состояние интерфейса на сервере. Общей будет база и специально созданные общие сервисы. Если первый компонент добавил сообщение в свой список, второй список от этого не изменится. Нужен отдельный путь уведомления второго компонента.

Поэтому «всё на сервере» не означает «всё в одном объекте». Компоненты разных пользователей изолированы по сеансам, а общение между ними мы проектируем явно. Это поможет дальше понять `singleton` уведомлений и `scoped` сервисы.

Вернуться по памяти: Сообщение уже сохранено в БД. Почему второй браузер ещё может его не показывать? Назови недостающий шаг.

Для проверки деталей: [1], [2]. Ссылки на с. 39-40.

02. HTTP: запрос и ответ

Ответ - конкретный объект с ограниченным временем жизни.

HTTP описывает обмен запросом и ответом. Запрос содержит метод, адрес, заголовки и иногда тело. GET /login просит страницу. POST /login передаёт данные формы. Ответ содержит статус, заголовки и тело: например HTML, JSON или пустое содержимое. Один и тот же браузер делает много таких обменов.

```
POST /login HTTP/1.1
Content-Type: application/x-www-form-urlencoded

Input.UserName=ivan&Input.Password=...
```

Это схематичный запрос: у реальной формы будут дополнительные поля, включая antiforgery-токен и сведения о форме. Для текущего объяснения важны имя пользователя и пароль в теле. Сервер читает их, проверяет и готовит новый ответ.

```
HTTP/1.1 302 Found
Set-Cookie: .AspNetCore.Identity.Application=...
Location: /
```

Заголовок Set-Cookie - инструкция браузеру сохранить cookie. Location - адрес перенаправления. Здесь именно браузер получает ответ, сохраняет cookie и выполняет следующий запрос. Статус и детали перенаправления в конкретной реализации могут отличаться; смысл обмена остаётся.

Заголовки должны быть отправлены до тела. Если сервер уже начал отдавать ответ, поздно менять статус или добавлять cookie. Ошибка «Headers are read-only, response has already started» указывает на момент отправки, а не на неправильный пароль.

HTTP-соединение может повторно использоваться для следующих запросов. Поэтому «ответ закончился» не означает «TCP-соединение обязательно закрыто». Для нас важнее другое: у законченного ответа больше нет возможности добавить заголовок.

Вернуться по памяти: В каком направлении идут Cookie и Set-Cookie? Почему это два разных заголовка?

Для проверки деталей: [3], [13]. Ссылки на с. 39-40.

03. Что делает браузер с HTML

HTML становится DOM, но не экземплярами C#-классов.

Браузер получает HTML и строит Document Object Model, или DOM: дерево узлов документа. Из DOM и CSS он вычисляет отображение. Поле `input` умеет принимать текст, кнопка умеет получать клик, ссылка умеет вести к адресу. Всё это существует независимо от Blazor.

JavaScript может подписываться на события DOM и изменять узлы. Скрипт Blazor использует эти возможности, чтобы связать документ с компонентами. Пользователь нажал кнопку - браузер породил событие `click` - скрипт выяснил, какой обработчик связан с событием, и инициировал нужное действие.

Файл `Room.razor` браузеру как исходный текст не нужен. Он не читает директивы `@inject` и `@code`. В Interactive Server соответствующий C# остаётся в серверном процессе. Браузер видит результат рендеринга и обслуживающий его JavaScript.

```
<input id="draft" />
<button>Send</button>
```

Даже при обрыве соединения HTML-поле может принимать символы. Его базовое поведение обеспечивает браузер. Но серверный обработчик `oninput` не выполнится, пока событие не будет доставлено. Поэтому возможность набрать текст и возможность обновить C#-поле компонента - разные вещи.

Обратная ситуация: серверное поле изменилось, а браузер ещё не получил обновление. В памяти сервера уже новое состояние, в DOM пока старое. Рендеринг и доставка изменений связывают эти состояния, но физически они находятся на разных машинах.

DOM - представление документа в браузере. Render tree - представление вывода компонента внутри Blazor. Это разные деревья с разными задачами.

Вернуться по памяти: При потере сети пользователь вводит букву в `input`. Где она появляется сразу? Где она может пока не появиться?

Для проверки деталей: [4], [5]. Ссылки на с. 39-40.

04. Тег не выбирает транспорт

Обычная кнопка может вызывать серверный C#.

В Razor смешаны HTML, компоненты и директивы. Название `button` обозначает HTML-элемент. Название `EditForm` обозначает Razor-компонент. Но сама категория тега не отвечает, отправится ли HTTP-запрос или событие Blazor. Это зависит от режима компонента и поведения элемента.

```
@rendermode InteractiveServer

<button @onclick="Increment">Add</button>
<p>@_count</p>

@code {
    private int _count;
    private void Increment() => _count++;
}
```

Здесь используется обычный `button`. Директива `@onclick` связывает `click` с C#-методом, а `Interactive Server` делает связь действующей через серверную интерактивность. Browser не исполняет `Increment`: он отправляет событие серверу.

Если тот же компонент отрендерить статически, обычный клик не вызовет этот C#-метод. Статический рендерер сформирует HTML, но не создаст интерактивную доставку кликов. Само наличие `@onclick` в исходнике ещё не гарантирует рабочую интерактивность.

Разметка и режим	Механизм
<code>button + @onclick</code> в <code>Interactive Server</code>	Событие через Blazor-соединение
<code>form method="post"</code> без перехвата	HTTP POST формы
<code>EditForm</code> в <code>static SSR</code>	HTTP POST и серверный обработчик
Ссылка с <code>forceLoad</code> / полная загрузка	Новый запрос документа

`PageTitle` управляет заголовком вкладки, `HeadOutlet` размещает соответствующий вывод в `head`, `Router` выбирает страницу. Их названия обозначают обязанности, а не специальные сетевые пакеты. В браузер не отправляется исполняемый тег `Router` с C# внутри.

Вернуться по памяти: Что нужно узнать про компонент, прежде чем объяснять поведение его `EditForm`?

Для проверки деталей: [4], [6]. Ссылки на с. 39-40.

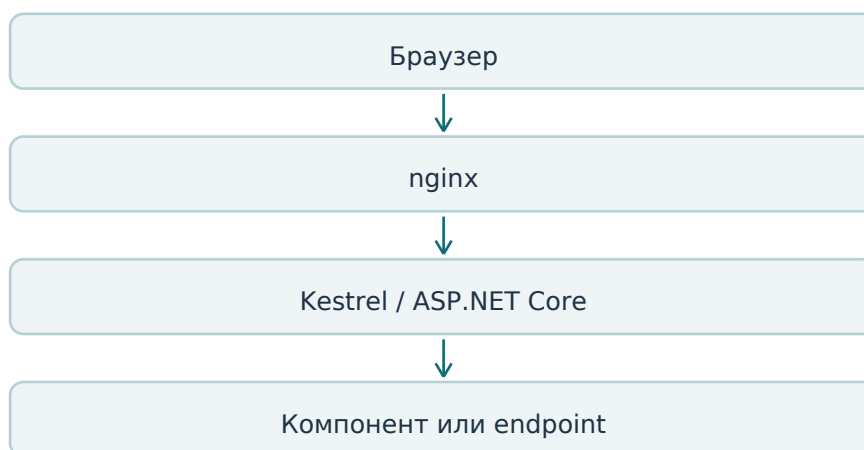
05. Процесс, Kestrel и приложение

Сервер - работающая программа, а не папка с endpoints.

При запуске RealTimeChat.BlazorServer операционная система создаёт процесс .NET. Выполняется Program.cs: загружается конфигурация, регистрируются сервисы, строится приложение. Затем хост запускает веб-сервер и ждёт входящие обращения. Одновременно процесс может обслуживать множество пользователей.

Kestrel - веб-сервер ASP.NET Core. Он принимает сетевые соединения и представляет HTTP-обмен приложению. Обработчик запроса получает HttpContext: сведения о запросе, ответе и связанных возможностях. Чат не создаёт Kestrel заново при каждом клике.

На компьютере разработчика браузер может обращаться прямо к Kestrel. При deploy перед ним часто стоит nginx. Nginx принимает внешний запрос, а затем пересылает его приложению. Это не новый экземпляр бизнес-логики: ChatService всё равно выполняется в процессе .NET.



Слово Server в имени проекта - наше соглашение. Если мы перенесли запуск в RealTimeChat.BlazorServer, именно этот проект стал серверным хостом. Само переименование не меняет протоколы и не создаёт отдельную архитектуру. Меняется содержимое приложения и его ответственность.

Процесс, поток, запрос и circuit нельзя считать синонимами. Один процесс содержит много объектов и обслуживает много запросов и circuits. Задачи выполняются на потоках по мере необходимости. Долгоживущее соединение не требует выделенного потока, который всё время ждёт конкретного пользователя.

Вернуться по памяти: Где работает ChatService после deploy: в nginx или в процессе .NET? Что делает nginx в этой цепочке?

Для проверки деталей: [7], [8]. Ссылки на с. 39-40.

06. Middleware и endpoints

Запрос проходит обработку до выбранной страницы.

Middleware - шаги HTTP-конвейера. Один шаг может перенаправить HTTP на HTTPS. Другой читает cookie и устанавливает пользователя. Следующий проверяет право доступа. Endpoint выполняет конкретную работу: возвращает API-ответ, запускает обработку формы или рендерит компонент.

```
app.UseAuthentication();
app.UseAuthorization();
app.UseAntiforgery();

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();
```

Порядок здесь имеет смысл. Сначала нужно узнать пользователя, затем проверить доступ. Antiforgery должен работать в подходящем месте после аутентификации и авторизации. В минимальном хостинге часть маршрутизации добавляется автоматически; видимый Program.cs не обязательно содержит весь внутренний конвейер.

Запрос /login и запрос CSS могут закончиться на разных endpoints. Middleware способен завершить обработку раньше: например вернуть ошибку или перенаправление. Поэтому не каждый запрос достигает LoginAsync или ChatService.

При нажатии интерактивной кнопки не возникает отдельный обычный запрос страницы, который заново проходит весь конвейер как POST /login. SignalR использует HTTP для установления и поддержки своего транспорта, но доставленное событие обрабатывается механизмом circuit. Authentication middleware не перечитывает браузерную cookie на каждом C#-событии.

Для отладки полезно ставить breakpoint в конкретном endpoint или lifecycle-методе. Если он не сработал, сначала выясни, действительно ли пришёл такой запрос. Возможно, произошло интерактивное событие, а возможно, доступ остановлен раньше.

Middleware отвечает за HTTP-обмен. Сервис приложения отвечает за сценарий. Эти уровни могут взаимодействовать, но их назначение различается.

Вернуться по памяти: Почему вызов UseAuthorization до распознавания пользователя мешает проверять доступ по его идентичности?

Для проверки деталей: [3], [13]. Ссылки на с. 39-40.

07. Add, Use, Map и Build

Четыре похожих группы вызовов выполняют разную работу.

В Program.cs легко увидеть одну длинную последовательность «магии». Раздели её на этапы. Add обычно регистрирует сервисы и настройки в DI. Build создаёт хост на основе собранной конфигурации. Use добавляет HTTP middleware. Map регистрирует endpoints или маршруты, в зависимости от конкретного метода. Run запускает хост.

```
builder.Services.AddApplication();
builder.Services.AddInfrastructure(connectionString);

var app = builder.Build();

app.UseAuthentication();
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();
app.Run();
```

AddApplication - наш extension method, а не встроенный закон архитектуры. Внутри него сейчас регистрируется ChatService. AddInfrastructure подключает DbContext, репозитории и Identity. Чтобы понять строку, нужно один раз открыть этот метод и увидеть фактические регистрации.

DI строит граф зависимостей. Компоненту нужен ChatService, сервису нужны репозитории, репозиториям нужен DbContext. Контейнер знает, как создать каждый узел. Если у SignInManager есть зависимость IAuthenticationSchemeProvider, а аутентификация не зарегистрирована, контейнер не сможет собрать сервис.

Именно это произошло в новом проекте: мы перенесли Infrastructure, но пропустили регистрацию аутентификации из старого хоста. Исключение возникло при проверке DI на Build, до первого обращения к PostgreSQL. Ошибка зависимостей и ошибка подключения к БД - разные этапы.

DI умеет создавать и освобождать зарегистрированные объекты. Он не проверяет автоматически, подходит ли время жизни каждого объекта для нашей логики.

Вернуться по памяти: Появилась ошибка Unable to resolve service for type X while attempting to activate Y. Какое звено отсутствует и где его искать?

Для проверки деталей: [9], [10]. Ссылки на с. 39-40.

08. Что такое Razor-компонент

Разметка и состояние становятся объектом .NET.

Файл `.razor` описывает компонент. При сборке Razor преобразует его в C#-код. Компонент имеет параметры, поля, `lifecycle`-методы и описание своего вывода. В `Interactive Server` экземпляры этого класса живут на сервере, а результат его рендеринга отображается в браузере.

```
@inject ChatService ChatService

<h1>@_roomName</h1>

@code {
    private string _roomName = "Lobby";
}
```

Поле `_roomName` не находится внутри HTML-тега `h1`. Оно находится в объекте компонента. При рендеринге Blazor читает значение и формирует вывод. Если поле позже изменится, нужно новое построение вывода, чтобы изменение стало видимым пользователю.

Компонент может содержать дочерние компоненты. Родитель передаёт параметры, дочерний показывает их или инициирует `callback`. В `server`-режиме это локальные C#-объекты одного сеанса. Каждый дочерний компонент не обязан создавать собственный `WebSocket`.

`Layout` - тоже компонент: он размещает тело страницы внутри общей оболочки. `Router` выбирает страницу, `RouteView` создаёт её представление. `_Imports.razor` добавляет директивы для области папок, но не является страницей, которую пользователь открывает.

Важно различать класс и экземпляр. `Room.razor` описывает один тип. Пользователи получают разные экземпляры с разными полями: у одного `RoomId` равен 1, у другого 2. `Static`-поле или `singleton`, напротив, может быть общим для многих пользователей. Ошибка с общим черновиком возникает не из-за Razor, а из-за неправильного размещения состояния.

Вернуться по памяти: Почему изменение обычного поля в экземпляре `Room` первого пользователя не изменяет `Room` второго?

Для проверки деталей: [11], [10]. Ссылки на с. 39-40.

09. Static SSR и обычная форма

С# может работать на сервере без интерактивного сеанса.

Static server-side rendering означает: в рамках запроса сервер выполняет компонент и формирует HTML. Браузер получает документ. Долгоживущая интерактивная связь с этим экземпляром компонента не создаётся. После завершения обработки запроса его поля не становятся сеансом интерфейса пользователя.

Это близко к нашему классическому PHP-чату. PHP выполнял скрипт, читал БД, формировал HTML, затем отдавал ответ. При следующем запросе работа начиналась заново. Razor использует другой язык и компонентную модель, но обычная последовательность «запрос - выполнение - ответ» знакома.

```
<EditForm Model="Input" method="post"
    FormName="login"
    OnValidSubmit="LoginAsync">
    ...
</EditForm>
```

В статической странице эта форма отправляется HTTP POST. Сервер создаёт обработку нового запроса, восстанавливает модель из присланных полей и вызывает подходящий обработчик. LoginAsync не запускается от удалённого click в старом экземпляре: он выполняется при серверной обработке формы.

Обработчик может вернуть страницу с ошибкой или перенаправить браузер. Здесь ему доступен актуальный HTTP-ответ, поэтому он способен выдать cookie. Не нужно постоянное соединение только ради проверки логина и пароля.

Слово static относится к отсутствию интерактивного С# после рендеринга. Оно не означает «данные неизменяемы» или «сайт без серверного кода». Сервер может получать данные из БД при каждом запросе, а HTML может содержать обычные формы, ссылки и собственный JavaScript.

Вернуться по памяти: Почему на static SSR странице можно выполнить LoginAsync через POST, но нельзя рассчитывать на обычный @onclick как на интерактивное событие?

Для проверки деталей: [6], [12]. Ссылки на с. 39-40.

10. Режим рендеринга и его область

Настройка возможностей приложения и выбор режима - разные шаги.

Blazor Web App позволяет размещать интерактивные части в разных средах. Для нашего проекта нужны static SSR для входа и Interactive Server для комнаты. В первом случае компонент обслуживает запрос. Во втором после начального отображения он участвует в живом серверном интерфейсе.

```
builder.Services.AddRazorComponents()  
    .AddInteractiveServerComponents();  
  
app.MapRazorComponents<App>()  
    .AddInteractiveServerRenderMode();
```

Эти строки делают серверную интерактивность доступной. Но они не означают, что каждая страница автоматически стала интерактивной. Выбор делается при назначении режима компоненту или более высокой границе дерева.

```
@page "/rooms/{RoomId:int}"  
@rendermode InteractiveServer
```

Так мы явно сделаем комнату интерактивной. Если назначить Interactive Server на Routes, Router распространяет этот режим на страницы: это глобальная интерактивность. Тогда нельзя просто вставить статический login внутрь интерактивного поддерева и ожидать независимый HTTP-сеанс.

В нашей начальной схеме мы убрали режим с Routes и HeadOutlet, чтобы страницы были статическими по умолчанию. Для приложения с интерактивными заголовками нужно также правильно настроить HeadOutlet: статический outlet не следует считать автоматически действующим участником интерактивного обновления PageTitle. Это отдельная граница рендеринга, которую надо согласовать с архитектурой страниц.

Другие режимы: Interactive WebAssembly выполняет компонент в браузере; Interactive Auto выбирает серверное выполнение при начальном использовании и клиентское при последующих посещениях после загрузки bundle. Auto не переносит уже работающий экземпляр с его полями из сервера в браузер посреди сеанса.

Вернуться по памяти: Почему наличия AddInteractiveServerComponents недостаточно, чтобы @onclick заработал на любой странице?

Для проверки деталей: [1]. Ссылки на с. 39-40.

11. Открытие интерактивной страницы

HTML появляется раньше, чем готов серверный обработчик клика.

Рассмотрим полную загрузку комнаты с включённым prerendering. Браузер запрашивает документ по адресу комнаты. Сервер выбирает страницу и сначала выдаёт HTML. Пользователь может увидеть заголовок и историю ещё до готовности интерактивности.

В документе подключён скрипт Blazor. Он запускается в браузере, находит интерактивные области и устанавливает связь с серверной частью. SignalR выполняет согласование транспорта; в обычных условиях используется WebSocket. При разрешённых настройках возможны другие транспорты, поэтому WebSocket не следует считать единственным логически допустимым вариантом.



Когда серверный интерактивный renderer готов, события из DOM можно доставлять компонентам. Кнопка уже видна во время prerender, но это ещё не гарантирует, что нажатие сможет вызвать серверный метод. Видимость и готовность к взаимодействию - разные этапы.

При prerender и интерактивном запуске используются разные экземпляры компонентов. Поэтому первоначальная загрузка данных может повториться. Значение, вычисленное в предварительном экземпляре, само по себе не обязано оказаться в полях интерактивного экземпляра.

У standalone WASM старт другой: браузер загружает runtime и сборки и запускает клиентское приложение. Но в обоих вариантах на экране есть HTML и JavaScript. Отличается место выполнения C# и способ синхронизации интерфейса.

В DevTools полная загрузка документа и установление Blazor-соединения видны как разные сетевые действия. Это удобный способ проверить модель на практике.

Вернуться по памяти: Пользователь видит кнопку, но приложение ещё подключается. Почему эти два факта не противоречат друг другу?

Для проверки деталей: [2], [14], [15]. Ссылки на с. 39-40.

12. Circuit: сеанс интерфейса

Circuit - не пользовательская cookie и не просто сокет.

Circuit связывает серверное состояние интерактивного UI с клиентом. В нём находятся `renderer`, экземпляры компонентов, связанные `scoped`-сервисы и сведения об аутентификации. Для обычной схемы одна вкладка создаёт собственный `circuit`. Второе окно с той же `cookie` не превращается в тот же экземпляр `Room`.

Соединение переносит сообщения. Circuit содержит состояние, ради которого эти сообщения передаются. Если сеть временно оборвалась, сервер может некоторое время удерживать отключённый `circuit`, чтобы клиент смог переподключиться. Поэтому конец транспортного соединения не всегда означает немедленное уничтожение всех компонентов.

Объект	Назначение
Cookie	Предъявить сведения об аутентификации
SignalR-соединение	Передавать сообщения
Circuit	Хранить и обслуживать серверный UI-сеанс
Компонент <code>Room</code>	Хранить состояние конкретного интерфейса

Если `circuit` утрачен, обычное восстановление связи не воссоздаёт автоматически все прежние поля компонента. Приложению может понадобиться новая загрузка и получение истории из БД. Дополнительные механизмы сохранения `circuit-state` в .NET 10 имеют свои ограничения и требуют настройки; это не обещание вечного хранения произвольной памяти.

Сервер платит памятью за активные и удерживаемые `circuits`. История на тысячу сообщений в каждом компоненте умножается на количество вкладок. Для маленького LAN-чата это обычно удобная модель. Для большой аудитории нужно измерять память и проектировать восстановление.

Circuit не является отдельным потоком. Framework координирует выполнение работы в его контексте. Пока пользователь ничего не делает, выделенный поток не обязан крутиться в ожидании.

Вернуться по памяти: Один пользователь открыл две вкладки. Что обычно общее: `cookie`, поле `_draft` или `circuit`?

Для проверки деталей: [14], [16]. Ссылки на с. 39-40.

13. Полный путь одного клика

От мыши до сервиса и обратно.

Пусть комната интерактивна, а кнопка связана с `SendAsync`. Клик возникает в DOM браузера. Скрипт Blazor получает событие и передаёт необходимые данные обработчика по соединению. Сервер сопоставляет сообщение с зарегистрированным обработчиком компонента.

```
private async Task SendAsync()
{
    var message = await ChatService.SendMessageAsync(
        RoomId, _userId, _draft, null,
        CancellationToken.None);
    _messages.Add(message);
    _draft = "";
}
```

Это упрощённый учебный фрагмент. Автор должен быть получен из проверенного состояния аутентификации, доступ к комнате должен быть разрешён, а отправка должна учитывать ошибки и отмену. Здесь показан только путь вызова: `ChatService` - репозиторий - EF Core - PostgreSQL.

Между компонентом и `ChatService` нет сериализации JSON или HTTP-запроса, если оба находятся в одном процессе и сервис внедрён через DI. Это обычные вызовы методов .NET. Между EF Core и PostgreSQL есть отдельный протокол базы данных.

После успешного сохранения обработчик меняет поля компонента. Blazor инициирует рендеринг, вычисляет изменения и отправляет их клиенту. Скрипт применяет их к DOM. Пользователь видит сообщение и очищенное поле.

Ни один шаг этой цепочки не обязан отправлять целиком `Room.razor` или заново весь HTML-документ. Кроме того, этот путь обновляет только текущего клиента. Другим пользователям требуется уведомление, которое вызовет аналогичное обновление их компонентов.

Вернуться по памяти: Назови две сетевые границы этого сценария. Почему компонент - `ChatService` не является третьей?

Для проверки деталей: [4], [5]. Ссылки на с. 39-40.

14. Рендеринг и изменение DOM

Изменить поле и показать новое значение - две операции.

Рендеринг компонента - вычисление его вывода на основе текущих параметров и полей. Blazor строит render tree, сравнивает подходящие участки с прежним выводом и формирует изменения. В Interactive Server результат доставляется клиенту в render batches, а клиентский скрипт применяет его к DOM.

Если ты написал `_count++`, состояние C# изменилось. Но браузер не умеет читать память сервера. Чтобы новое число появилось, renderer должен обработать компонент, а изменения должны доехать до клиента. В этом смысле `StateHasChanged` означает запрос рендеринга, а не прямую запись в браузер.

```
private void Increment()
{
    _count++;
    // Для обычного обработчика Blazor
    // framework сам инициирует рендеринг.
}
```

После обычных событий framework обычно сам инициирует обновление. Поэтому добавлять `StateHasChanged` после каждого присваивания не нужно. Однако событие внешнего сервиса или таймер не обязательно является событием Blazor: такой путь требует явного уведомления renderer и правильного контекста выполнения.

Несколько запросов рендеринга могут объединяться. Повторный рендеринг не обязательно заменяет каждый DOM-узел. Это важно для сохранения фокуса, состояния элементов и производительности. В списках `@key` помогает связать элементы с устойчивой идентичностью, но не решает все проблемы сортировки и конкурентных изменений.

Не путай рендеринг с повторным чтением БД. Сам рендер читает состояние компонента. Если запрос к БД находится в getter, вызываемом разметкой, ты сам создаёшь скрытую работу на каждом отображении. Обычно данные загружают в явных lifecycle-методах или обработчиках.

Вернуться по памяти: Почему `StateHasChanged` не является командой «сходить в базу и обновить данные»?

Для проверки деталей: [5]. Ссылки на с. 39-40.

15. Жизненный цикл компонента

У каждого метода свой повод для выполнения.

`OnInitializedAsync` подходит для начальной подготовки экземпляра.

`OnParametersSetAsync` вызывается при поступлении параметров: например `RoomId` поменялся при переходе в другую комнату. `OnAfterRenderAsync` выполняется после интерактивного рендеринга, когда доступны ссылки на DOM для JS interop. `Dispose` освобождает ресурсы компонента.

Метод	Что удобно делать
<code>OnInitializedAsync</code>	Начальная настройка экземпляра
<code>OnParametersSetAsync</code>	Загрузка данных для нового <code>RoomId</code>
<code>OnAfterRenderAsync</code>	Прокрутка, DOM-зависимый JS interop
<code>Dispose / DisposeAsync</code>	Отписка и освобождение ресурсов

Изменение адреса с `/rooms/1` на `/rooms/2` не гарантирует новый экземпляр того же типа страницы. Framework может использовать текущий экземпляр и передать ему новый `RoomId`. Поэтому история, зависящая от параметра, должна реагировать на параметры, а не только на первоначальную инициализацию.

`OnAfterRenderAsync` не выполняется во время static SSR и prerendering. На сервере тогда нет живого DOM конкретного браузера, который можно прокрутить. Поэтому доступ через `ElementReference` и вызов `scrollChatToBottom` планируют на интерактивную фазу после рендеринга.

Асинхронный lifecycle-метод может дать промежуточный вывод «Loading...». Поля до завершения `await` должны быть в состоянии, которое разметка умеет обработать. Не предполагая, что `_room` всегда уже загружена, мы предотвращаем `NullReferenceException`.

При уничтожении компонента нужно убрать подписки на общий сервис. Иначе сервис может продолжать держать ссылку на ушедшую страницу и вызывать её обработчики. Освобождение - часть поведения приложения, а не косметика.

Вернуться по памяти: Куда поместить загрузку комнаты по `RoomId` и почему одного `OnInitializedAsync` недостаточно?

Для проверки деталей: [11], [17]. Ссылки на с. 39-40.

16. Prerendering и двойная инициализация

Два вызова могут принадлежать двум экземплярам.

Prerendering даёт пользователю HTML до запуска интерактивности. Это улучшает первое отображение, но требует учитывать две фазы. Сначала серверный рендерер обслуживает HTTP-запрос. Затем создаётся интерактивный экземпляр для circuit. Начальная инициализация может произойти в обеих фазах.

Если поставить лог в `OnInitializedAsync` и увидеть его дважды, это не обязательно ошибка и не обязательно один объект, который framework неожиданно переинициализировал. Добавь идентификатор экземпляра и увидишь, какие экземпляры участвуют в каждом этапе.

```
private readonly Guid _instanceId = Guid.NewGuid();

protected override Task OnInitializedAsync()
{
    Console.WriteLine($"Init: {_instanceId}");
    return Task.CompletedTask;
}
```

Повторное чтение истории может быть допустимо в маленьком чате. Повторная запись сообщения, регистрация пользователя или необратимое действие в инициализации - уже опасный дизайн. Lifecycle запуска страницы должен в первую очередь готовить интерфейс, а не выполнять случайный бизнес-сценарий.

Возможные решения повторной загрузки: передать подготовленное состояние через `PersistentComponentState`, кэшировать подходящие данные или отключить prerender для конкретной интерактивной области. Выбор зависит от того, нужны ли ранний HTML и единая начальная загрузка.

```
@rendermode @(new InteractiveServerRenderMode(
    prerender: false))
```

Это вариант настройки, а не обязательная правка нашего проекта. С отключённым prerender интерфейс ждёт интерактивного подключения. Не стоит отключать механизм только потому, что два лог-сообщения выглядят непривычно: сначала выясни, что именно повторяется и имеет ли это последствия.

Вернуться по памяти: Почему сохранение отправленного сообщения в `OnInitializedAsync` хуже, чем повторное чтение списка комнат?

Для проверки деталей: [15], [11]. Ссылки на с. 39-40.

17. await и повторный вход в компонент

Один логический контекст не устраняет все гонки.

Blazor координирует выполнение в рамках circuit через synchronization context. Обычно одновременно внутри одного circuit исполняется один участок работы. Но await освобождает выполнение до готовности результата. За это время может произойти другое событие, изменение параметров или уничтожение компонента.

Пользователь открывает комнату 1. Начинается запрос истории. Пока он ждёт БД, пользователь переходит в комнату 2. Начинается второй запрос. Если первый завершится позже и без проверки присвоит _messages, на странице комнаты 2 окажется история комнаты 1. Это логическая гонка, даже если присваивания никогда не выполнялись одновременно на разных потоках.

```
var requestedRoomId = roomId;  
var messages = await LoadMessagesAsync(requestedRoomId);  
  
if (roomId != requestedRoomId)  
    return;  
  
_messages = messages;
```

Проверка демонстрирует принцип. Для последовательности 1 - 2 - 1 одного roomId уже недостаточно: старый запрос к комнате 1 может совпасть с текущим адресом. Тогда используют счётчик версии загрузки, отмену или другой способ отличить старую операцию от новой.

Ещё один пример: две отправки подряд. Пока первая ждёт сохранения, вторая может начаться. Флаг _isSending и disabled-кнопка помогают избежать случайного повторения, но корректность сервиса и БД не должна зависеть только от визуального запрета.

Не блокируй circuit через .Result или .Wait(). Используй await. Если событие пришло от внешнего сервиса или таймера, перенос изменения UI в InvokeAsync нужен для согласования с renderer, а не для создания отдельного HTTP-запроса.

Вернуться по памяти: Почему отсутствие одновременного выполнения двух присваиваний не исключает ситуацию с устаревшей историей?

Для проверки деталей: [18]. Ссылки на с. 39-40.

18. Scoped: на запрос или на circuit?

Слово `scoped` не означает «короткоживущий».

В обычном API `scoped`-сервис часто создаётся на HTTP-запрос и освобождается после его завершения. В `Interactive Server` стандартный DI scope связан с `circuit`. Поэтому один `ChatService`, внедрённый в несколько компонентов `circuit`, может жить заметно дольше одного клика.

Lifetime	Обычное значение
Singleton	Один экземпляр на DI-контейнер хоста
Scoped в HTTP-обработке	Один экземпляр в scope запроса
Scoped в circuit	Один экземпляр в scope circuit
Transient	Новый экземпляр при разрешении зависимости

В статической фазе и `circuit` существуют разные scopes. Сервис, полученный во время `prerender`, не следует считать тем же объектом, который затем внедрён в интерактивную страницу. Граница режима означает также границу некоторых зависимостей и состояния.

Singleton уведомлений удобно видеть всем компонентам, но он не должен хранить текущий пароль или «текущего пользователя» в одном поле. Пользователей много. Если `singleton` хранит присутствие, записи должны иметь явные ключи пользователя и участия.

Transient не гарантирует автоматическую очистку после каждого вызова метода. Это означает новый объект при разрешении. Если его создал контейнер и он `IDisposable`, контейнер может удерживать его для освобождения до завершения соответствующего scope. Поэтому смена lifetime не заменяет проектирование ownership.

Нельзя внедрять `scoped ChatService` прямо в `singleton` уведомлений и использовать как универсальный сервис всех пользователей. Получится захват зависимости с неподходящей жизнью. Общий `notifier` передаёт события; операции с БД выполняются в корректно созданном scope или через фабрику.

Вернуться по памяти: Почему `AddScoped<ChatService>()` ведёт себя по-разному в endpoint API и в интерактивном компоненте?

Для проверки деталей: [10], [9]. Ссылки на с. 39-40.

19. DbContext в серверном UI

Рабочий список комнат ещё не доказывает правильный lifetime.

Наши репозитории получают ApplicationDbContext через DI. При AddDbContext он обычно scoped. В старом API такой scope был коротким запросом. При прямом внедрении ChatService в интерактивную страницу тот же подход может держать DbContext всю жизнь circuit.

DbContext хранит сведения о tracked-объектах и выполняемых операциях. Он не потокобезопасен и не допускает произвольные перекрывающиеся запросы. Две компоненты или повторная операция во время await могут начать работу через один context. Возможны исключения о второй операции и накопление ненужного состояния.

AsNoTracking уменьшает tracking для чтения, но не делает DbContext безопасным для конкурентных операций. Флаг загрузки помогает одному экрану, но не обязательно защищает все компоненты, которые используют общий scoped context.

Два подхода для нашего приложения

Первый: создавать DI scope на отдельную операцию. Из этого scope получать ChatService, ждать завершения сценария и освобождать scope. Его репозитории разделяют один DbContext в рамках сценария, но следующий вызов получит новый. Компонент не должен хранить сервис из уже освобождённого scope.

Второй: использовать IDbContextFactory и создавать context в операции. Тогда нужно согласовать репозитории и границу транзакции: несколько context в одном сценарии не означают общую атомарную запись. Нельзя просто заменить регистрации, не проверив ownership и связи зависимостей.

```
await using var scope = ScopeFactory.CreateAsyncScope();
var chat = scope.ServiceProvider
    .GetRequiredService<ChatService>();

var rooms = await chat.GetRoomsAsync(cancellationToken);
```

Это учебный фрагмент первого подхода; нужны внедрённый IServiceScopeFactory и using соответствующего namespace. Возвращай DTO или независимые данные, а не query или объект, которому ещё нужен context после освобождения.

Вернуться по памяти: Почему список комнат мог работать, хотя тот же DI-подход позднее сломается при отправке и уведомлениях?

Для проверки деталей: [19]. Ссылки на с. 39-40.

20. Как обновить второго пользователя

Сохранение данных и уведомление UI решают разные задачи.

Первый пользователь отправил сообщение, ChatService сохранил его в PostgreSQL. Эта запись долговечна, но второй компонент о ней не узнает сам. Запрос к БД не подписывает страницу на будущие вставки. Мы создадим общий механизм уведомлений внутри процесса.

Сохранение → публикация → обновление UI

После успешного сохранения notifier сообщает о новом сообщении подписчикам. Каждый компонент решает, относится ли сообщение к его комнате и доступно ли пользователю. Для приватного сообщения видимость ограничена автором и получателем. Полезнее фильтровать адресатов до доставки данных обработчикам, а проверку в компоненте использовать как дополнительную защиту.

```
private Task OnMessageAsync(ChatMessageInfo message)
{
    return InvokeAsync(() =>
    {
        if (message.RoomId != RoomId) return;
        _messages.Add(message);
        StateHasChanged();
    });
}
```

Фрагмент предполагает, что доступ к приватным данным уже проверен при доставке. InvokeAsync переводит работу в контекст UI, а StateHasChanged просит рендер. Blazor затем сам доставляет изменения DOM браузеру. Notifier не пишет непосредственно в внутренний сокет Blazor.

При уходе со страницы компонент отписывается. Ошибка одного подписчика не должна мешать уведомлять остальных. Обычное многоадресное событие с Func<Task> требует осторожности: наивный Invoke возвращает только последний результат. Поэтому уведомления нужно явно перечислять и ожидать согласно выбранной политике.

Notifier в памяти одного процесса не передаёт события другим серверам. И при сбое после записи в БД push может потеряться. История остаётся источником восстановления; для гарантированной доставки нужны дополнительные механизмы, например outbox. Для учебного однопроцессного чата достаточно понимать границу.

Вернуться по памяти: Почему notifier должен публиковать сообщение после успешного сохранения, а не перед ним?

Для проверки деталей: [18], [17]. Ссылки на с. 39-40.

21. ChatHub и внутренний Hub Blazor

Одинаковая библиотека, разные протоколы приложения.

В WASM-версии у нас был собственный ChatHub. Мы определяли `JoinRoom`, `SendMessage`, `NotifyTyping` и события `ReceiveMessage`. Клиент создавал `HubConnection` и вызывал эти методы. SignalR обслуживал транспорт и протокол вызовов, а смысл сообщений задавали мы.

У Interactive Server тоже есть SignalR-инфраструктура, но её использует framework для UI: передачи событий, `render batches` и обслуживания `circuit`. Это не наш ChatHub с методами чата. Для обновления server-компонента мы меняем состояние и иницилируем рендеринг.

Собственный ChatHub	Blazor UI-соединение
Наши методы и события	Внутренние сообщения framework
Клиент вызывает <code>SendMessage</code>	Клиент передаёт DOM-событие
<code>ReceiveMessage</code> содержит DTO чата	Render batch содержит изменения UI
Мы организуем группы чата	Framework обслуживает <code>circuit</code>

Hub не является одним постоянно работающим объектом всей программы. Экземпляры Hub краткосрочные; SignalR создаёт их для обработки вызовов. Поэтому состояние подключения мы раньше держали в `Context.Items` или отдельном `RoomPresence`, а не в обычном поле одного экземпляра ChatHub.

В новой схеме компоненты живут дольше и получают общие уведомления. Собственный ChatHub для интерфейса чата можно убрать. При этом SignalR как часть механизма Blazor остаётся. Поэтому «чат без ChatHub» не означает «чат без постоянной связи с браузером».

Так мы убираем одну прикладную сетевую границу и оставляем framework заботиться об интерфейсной. Если позже понадобятся мобильный клиент или другой frontend, API и прикладной Hub снова могут стать полезны.

Вернуться по памяти: Что должен сделать компонент после уведомления о сообщении: вызвать `Clients.SendAsync` или изменить состояние и запросить рендеринг?

Для проверки деталей: [14], [20]. Ссылки на с. 39-40.

22. Cookie: кто хранит и кто проверяет

Браузер переносит билет; сервер устанавливает идентичность.

После проверки пароля Identity создаёт ClaimsPrincipal: объект со сведениями о пользователе, например ID и именем. Cookie-аутентификация защищает authentication ticket и выдаёт его браузеру. Браузер хранит cookie и отправляет её с подходящими HTTP-запросами согласно домену, пути и атрибутам.

При следующем запросе authentication handler читает cookie, проверяет защиту и срок действия и восстанавливает principal. Затем приложение может узнать User.Identity.Name и claim с идентификатором. Cookie не должна содержать открытый пароль или его хеш. Хеш хранится на стороне Identity для проверки входа.

Наша cookie обычно содержит защищённый билет со сведениями, а не просто номер строки «сеанса» в таблице. Возможны настройки с серверным хранилищем tickets, но это отдельная схема. Нельзя считать все cookie-аутентификации одинаковыми по устройству.

```
builder.Services
    .AddAuthentication(IdentityConstants.ApplicationScheme)
    .AddIdentityCookies();
```

Выбор application scheme связывает проверку запросов с нужным обработчиком. AddIdentityCookies регистрирует cookie-схемы Identity. SignInManager использует их при входе. Одной регистрации UserManager и репозитория недостаточно для полного механизма.

Аутентификация отвечает «кто это». Авторизация отвечает «что ему разрешено». Успешный вход не означает право читать чужое приватное сообщение. Cookie узнаёт пользователя; правила доступа определяют, какие сообщения этому пользователю можно вернуть.

Authentication ticket защищается механизмом Data Protection. После deploy сохранность и совместимость ключей влияют на возможность читать ранее выданные cookies.

Вернуться по памяти: Почему knowing userId из cookie не даёт автоматически право читать все сообщения этого пользователя и других людей?

Для проверки деталей: [13], [21], [22]. Ссылки на с. 39-40.

23. Почему интерактивный вход ломается

Место выполнения C# не создаёт новый HTTP-ответ.

В интерактивной комнате `button` вызывает C# на сервере. Это создаёт соблазн вызвать `PasswordSignInAsync` прямо из такого обработчика. Но `SignInManager` при успешном входе должен выдать cookie: изменить заголовки ответа, который попадёт именно в браузер пользователя.

Событие Blazor приходит по уже установленной связи. Исходный HTTP-ответ страницы давно отправлен. В обработке DOM-события нет нового обычного ответа страницы, в который можно безопасно добавить `Set-Cookie`. Даже если какой-то `HttpContext` доступен, он не становится правильным изменяемым ответом нового запроса входа.

Обычный POST /login	Интерактивный click
Новый запрос и ответ	Сообщение в существующей связи
Можно подготовить <code>Set-Cookie</code> до ответа	Нельзя дописать заголовки старого ответа
После ответа браузер получает cookie	UI-сообщение не выдаёт cookie само

Отсюда ошибка «Headers are read-only». Её причина не в кнопке HTML, не в Razor-теге и не в неправильном DI lifetime самого компонента. Мы выбрали неподходящий канал для операции, которая должна изменить браузерную аутентификацию через HTTP.

Также нельзя решить задачу обычным серверным `HttpClient`, отправив запрос входа самому себе. Ответ с `Set-Cookie` тогда получает `HttpClient`, а не браузер. Чтобы cookie оказалась у пользователя, её должен получить браузер в своём HTTP-обмене.

Варианты решения: статическая SSR-форма, Razor Page или HTTP endpoint, к которому обращается браузер. Наш выбор - статическая SSR-форма, потому что она сохраняет привычную Razor-разметку и позволяет явно увидеть полный обмен.

Вернуться по памяти: Почему серверный `HttpClient`, успешно выполнивший login, ещё не означает вход пользователя в браузере?

Для проверки деталей: [23], [13]. Ссылки на с. 39-40.

24. Вход по шагам

Один сценарий, две загрузки и одна новая cookie.

Сначала браузер делает GET /login. Сервер формирует HTML формы. Данные модели пока пусты. Пользователь вводит имя и пароль в браузере; при static SSR C#-модель на сервере не обновляется после каждого символа. Её заполнит следующий POST.

Браузер отправляет форму. Сервер обрабатывает POST, сопоставляет форму и заполняет Input. DataAnnotationsValidator проверяет Required. Если модель валидна, вызывается OnValidSubmit. SignInManager ищет пользователя, проверяет пароль и правила входа, а при успехе готовит cookie.

```
var result = await SignInManager.PasswordSignInAsync(
    Input.UserName.Trim(), Input.Password,
    isPersistent: false, lockoutOnFailure: true);

if (result.Succeeded)
    Navigation.NavigateTo("/", forceLoad: true);
```

Здесь isPersistent: false означает сеансовую cookie с точки зрения браузерного хранения; срок допустимости билета всё равно задаётся настройками. lockoutOnFailure включает учёт неудачи для механизма блокировки, если соответствующие правила и пользователь позволяют блокировку.

Перенаправление в static SSR обрабатывается framework как часть ответа. При BlazorDisableThrowNavigationException в .NET 10 NavigateTo не обязан прерывать выполнение исключением. Если после него есть код, явно заверши ветку return, когда дальнейшая работа не нужна.

После ответа браузер уже хранит cookie и загружает /. Новый запрос содержит её. Сервер показывает имя пользователя. При последующем запуске интерактивной комнаты circuit получает состояние аутентификации. Неверный пароль даёт страницу с сообщением, без успешной cookie входа.

Регистрация нового пользователя - отдельный сценарий. Для проверки этого этапа используем уже существующих пользователей нашей PostgreSQL-базы.

Вернуться по памяти: В какой момент текст из HTML-input становится Input.UserName в серверном компоненте static SSR?

Для проверки деталей: [12], [13], [24], [25]. Ссылки на с. 39-40.

25. Binding, валидация и antiforgery

Три механизма с разными обязанностями.

В интерактивной форме @bind обновляет модель через доставку событий. В static SSR ввод сначала существует только в HTML-полях браузера. При POST сервер восстанавливает модель из присланных значений. SupplyParameterFromForm отмечает свойство для такого заполнения, а FormName помогает выбрать нужную форму.

```
[SupplyParameterFromForm]
private LoginInput Input { get; set; } = default!;

protected override void OnInitialized()
{
    Input ??= new();
}
```

Оператор ??= важен: при первоначальном GET создаётся пустая модель, а при POST уже заполненную модель нельзя затереть новым объектом. Если каждый раз писать Input = new(), обработчик может потерять значения, которые только что прислал пользователь.

DataAnnotationsValidator проверяет правила вроде Required. Он не проверяет, совпадает ли пароль с хешем в базе: это задача Identity. OnValidSubmit означает, что выполнены правила модели формы, а не что пользователь уже успешно вошёл.

Antiforgery защищает от CSRF: чужой сайт пытается заставить браузер отправить действие с cookie нашего сайта. Authentication cookie может прикладываться браузером автоматически; сам факт её наличия не доказывает, что пользователь намеренно отправил нашу форму.

EditForm автоматически включает antiforgery-токен для серверных форм в Blazor Web App. Для обычной HTML-формы применяется AntiforgeryToken согласно настройке обработчика. Middleware проверяет соответствующие данные. Он не заменяет проверку пароля, права доступа, HTTPS или защиту от XSS.

Наш login использует POST и token-защиту. Если после входа старая вкладка отправляет форму с токеном, созданным для другого состояния пользователя, возможна ошибка проверки. Полная загрузка заново получает форму в актуальном состоянии. Не нужно исправлять такую ошибку отключением защиты.

Вернуться по памяти: Почему успешная валидация Required не означает успешную аутентификацию? Что проверяет antiforgery вместо пароля?

Для проверки деталей: [12], [6], [13]. Ссылки на с. 39-40.

26. Пользователь внутри circuit

AuthorizeView отображает состояние, но не выдаёт cookie.

AuthenticationStateProvider предоставляет компонентам состояние аутентификации: ClaimsPrincipal пользователя. AddCascadingAuthenticationState делает его доступным в дереве компонентов. AuthorizeView выбирает отображаемую ветку по этому состоянию.

```
<AuthorizeView>
  <Authorized Context="auth">
    <p>@auth.User.Identity?.Name</p>
  </Authorized>
  <NotAuthorized>
    <a href="/login">Sign in</a>
  </NotAuthorized>
</AuthorizeView>
```

AuthorizeView не проверяет пароль и не выписывает cookie. Он потребляет уже установленное состояние. Если circuit создан для анонимного пользователя, выдача cookie в другом HTTP-обмене не должна считаться автоматическим мгновенным изменением всех его компонентов.

Новый полный вход в приложение создаёт корректную исходную связь между cookie, principal и UI-сеансом. Для долгих circuits может потребоваться повторная проверка Identity security stamp. Стандартный Identity-шаблон предоставляет соответствующий provider; наш минимальный хост не получает всю эту логику только от AddCascadingAuthenticationState.

Не используй HttpContext.User как универсальный источник «текущего пользователя» в каждом интерактивном обработчике. HttpContext относится к HTTP, может отсутствовать или не отражать актуальное circuit-состояние. Для компонентов используем предназначенный для них механизм аутентификации.

AuthorizeView помогает интерфейсу, но скрытая кнопка не является полной защитой операции. На сервере должны проверяться права сценария и видимость данных. Для страниц применяются Authorize и правильная настройка маршрутизации; для приватных сообщений нужны правила в обработке данных.

Имя и ID автора берём из проверенного principal. Не доверяем полю authorId, которое пользователь может прислать в форме.

Вернуться по памяти: Почему «на экране нет кнопки» слабее, чем «сервер не разрешает выполнить операцию»?

Для проверки деталей: [13]. Ссылки на с. 39-40.

27. Выход и изменение аутентификации

Удалить cookie и прекратить сеанс UI - разные действия.

Выход тоже меняет cookie браузера. SignOutAsync готовит ответ, который удаляет или истекает authentication cookie. Для этого нужен HTTP-обмен с браузером. Поэтому logout удобно реализовать как защищённый POST, а после него полностью загрузить анонимную страницу.

Не делай выход обычной ссылкой GET только ради удобства. Ссылка может быть предварительно загружена или открыта без намерения выполнить действие. POST с antiforgery яснее отделяет изменение состояния от чтения страницы.

В уже работающем circuit остаётся установленное состояние пользователя. Исчезновение cookie в браузере само по себе не означает немедленную остановку всех старых circuits. Например, пользователь вышел в одной вкладке, а другая пока показывает прежний интерфейс. Это причина проектировать revalidation и критические проверки доступа.

Действие	Что меняется
SignOutAsync в HTTP-обработке	Браузерная cookie через ответ
Полная загрузка после выхода	Новый запрос и новый UI-сеанс
Повторная проверка Identity	Актуальность права оставаться вошедшим
Проверка бизнес-операции	Право на конкретное действие

После удаления пользователя или смены пароля старые подключения тоже требуют внимания. Политика может быть разной: периодическая проверка, завершение определённых сеансов или обязательная проверка перед чувствительной операцией. Учебный чат не обязан сразу иметь корпоративный session management, но важно понимать, что минимальный login не решает эту задачу автоматически.

Для практической реализации следующего шага мы отдельно напишем обработчик выхода и проверим две вкладки. Это будет конкретная проверка границы между HTTP-аутентификацией и долгоживущим UI.

Вернуться по памяти: Почему после выхода в одной вкладке другая может ещё показывать имя пользователя?

Для проверки деталей: [13], [21]. Ссылки на с. 39-40.

28. Навигация: адрес меняется по-разному

Изменение URL не всегда означает новый документ.

Браузер может полностью загрузить адрес, а может выполнить клиентскую навигацию. При интерактивной маршрутизации Router выбирает новую страницу внутри уже работающего приложения. Страница меняется, но circuit может сохраниться. Поэтому переход между комнатами не равен перезапуску хоста.

При static SSR enhanced navigation может получить HTML нового адреса через fetch и обновить документ без полной загрузки. Это всё ещё HTTP-запрос к серверу. Видимое отсутствие перезагрузки не доказывает, что сработало интерактивное сообщение Blazor.

```
Navigation.NavigateTo("/login", forceLoad: true);
```

forceLoad просит полную загрузку. Это полезно при переходе между режимами и при изменении cookie-состояния. Но смысл вызова зависит от места: в обработке static SSR навигация оформляется сервером в HTTP-ответе, в интерактивном компоненте команда приходит клиентскому механизму навигации.

Если Routes глобально интерактивен, login должен быть корректно исключён из интерактивной области через поддерживаемую схему статических страниц. В нашем простом варианте Routes статический, а комната явно интерактивная. Это упрощает начальное разделение, но настройку заголовков и навигации всё равно нужно проверять.

При переходе roomId может измениться у того же экземпляра. Компоненту нужно очистить выбранного получателя, отменить старую загрузку, обновить подписку на комнату и получить новую историю. Сам адрес не выполнит все эти прикладные действия за нас.

Чтобы понять навигацию, смотри DevTools: был ли запрос документа, fetch HTML или сообщение внутри соединения? Одинаковый результат на экране может быть получен разными путями.

Вернуться по памяти: Какие два разных механизма могут поменять страницу без полной загрузки браузера?

Для проверки деталей: [24], [1]. Ссылки на с. 39-40.

29. Архитектура нашего Blazor Server чата

Один хост, прямые сценарии, явная доставка событий.

RealTimeChat.BlazorServer содержит запуск, страницы, вход и серверные механизмы UI. Application содержит ChatService и абстракции репозитория. Domain содержит сущности и их правила. Infrastructure выполняет хранение через EF Core и подключает Identity. Отдельный HTTP API для своих компонентов нам не требуется.

Часть	Обязанность
Компонент Room	UI, черновик, отображение, события
ChatService	Сценарии чтения и отправки
Репозитории / EF Core	Доступ к PostgreSQL
Notifier	Доставка событий подписчикам
Presence	Учёт активных участников в комнатах

Компонент получает проверенного пользователя и выполняет разрешённый сценарий. Для операций с БД используем подходящую границу `scope`. После сохранения сообщение публикуется `notifier`. Подписчики обновляют свои списки через `InvokeAsync`. Framework доставляет обновления браузерам.

Presence хранит участия, а не только список уникальных `userId`. Один человек может открыть две вкладки. Если закрыть одну, нельзя объявить человека офлайн, пока второе участие активно. Ключ участия может быть собственным ID сеанса или компонента; нельзя бездумно переносить `connectionId` старого ChatHub в новую модель.

Typing - временное состояние. Для него обычно достаточно события и тайм-аута, а не записи в `messages`. История сообщений долговечна; присутствие и `typing` зависят от живых сеансов. Задержка удаления `presence` при кратком обрыве может быть разумной, чтобы список не мигал при `reconnect`.

В private messaging фильтруем историю по пользователю и адресуем `push` только разрешённым участникам. Проверка в браузере не подходит как защита. Компоненты находятся на сервере, но общая подписка на все события тоже не освобождает нас от правил доступа.

Вернуться по памяти: Какие данные должны пережить перезапуск сервера, а какие естественно построить заново?

Для проверки деталей: [19], [18]. Ссылки на с. 39-40.

30. Server и WASM: выбирать по условиям

Масштаб функциональности не определяет место исполнения.

Blazor Server уменьшает прикладные сетевые слои: компонент вызывает сервис, сервер хранит UI-состояние, framework обслуживает события. Это удобно для нашего учебного чата. Цена - память под circuits и зависимость серверных действий интерфейса от соединения и задержки.

WASM переносит C# компонентов в браузер. Локальная фильтрация, редактирование и часть валидации могут работать без серверного round trip. Но загрузка runtime и сборок требует времени, а сохранение данных и проверка прав всё равно выполняются через backend. Клиентское выполнение не делает бизнес-логику доверенной.

Условия	Что рассмотреть
Внутренний UI, стабильная сеть	Interactive Server
Много локальных операций и черновиков	WASM или другой клиентский UI
Каталог для первой загрузки и поиска	Серверный HTML / SSR
Много активных вкладок по миру	Измерить задержку и стоимость circuits
Мобильный клиент рядом с web	Явный API для нескольких клиентов

В CRM редактор документов и таблица с формулами дают настоящую клиентскую логику: выделение, undo, локальные вычисления, промежуточное состояние. WASM позволяет писать её на C#, но зрелые JS-компоненты и браузерные API всё равно могут потребовать interop.

Marketplace не обязан целиком быть WASM. Каталог можно отдавать серверным HTML, кабинеты сделать интерактивными, а сложный редактор разместить на клиенте. Границы выбираются по требованиям, а не по ярлыку «enterprise».

Чат требует сети для доставки, но не обязательно для всех локальных действий. В WASM можно продолжать обрабатывать черновик при обрыве; в Server обычное поле принимает текст, но серверные C#-действия ждут связи. Оба варианта допустимы с разными компромиссами.

Вернуться по памяти: Назови требование, которое склоняет к WASM, не используя фразу «приложение большое».

Для проверки деталей: [2], [1]. Ссылки на с. 39-40.

31. Deploy, reconnect и несколько серверов

Простая локальная схема имеет конкретные границы.

В нашем deploy планируется один процесс .NET на Ubuntu, PostgreSQL и nginx перед приложением. Nginx должен корректно пропускать транспорт Blazor, а приложение должно знать нужные сведения прокси и внешнюю схему HTTPS. Обычный успешный GET ещё не доказывает, что интерактивное соединение работает.

Перезапуск процесса уничтожает его обычную память: компоненты, notifier, presence и неперсистентное состояние circuits. Сообщения в PostgreSQL остаются. Клиент должен иметь путь восстановления: переподключение, а при невозможности - новая загрузка и чтение истории.

Нужно сохранять Data Protection keys подходящим способом. Если ключи потеряны, приложение может перестать читать ранее выданные authentication cookies и antiforgery-данные. В нескольких экземплярах приложения совместимость ключей и application name становится отдельной задачей.

При двух процессах singleton notifier существует в каждом отдельно. Сообщение, сохранённое процессом А, не вызывает автоматически обработчик в процессе В. Понадобится межпроцессная доставка событий или другой общий механизм. Для корректного размещения stateful circuits обычно нужна affinity; backplane SignalR сам по себе не переносит все объекты circuit на другой сервер.

В WASM клиентский UI не хранится в памяти сервера, но прикладной realtime-chat всё равно может требовать постоянных соединений и общего присутствия. Перенос интерфейса в браузер не отменяет проектирование серверного распределённого состояния.

Для учебного проекта один процесс - сознательная достаточная граница. Распределённую доставку не добавляем заранее, но описываем ограничение в README, чтобы через год понимать, почему notifier хранится в памяти.

Вернуться по памяти: Почему добавление второго сервера за nginx не заставит два singleton notifier стать одним общим объектом?

Для проверки деталей: [14], [16], [22]. Ссылки на с. 39-40.

32. Как исследовать механизм самостоятельно

Проверять один маршрут действия за раз.

DevTools показывает браузерную сторону, debugger - серверную. Открой Network, сохрани лог и перейди на /login. Найди запрос HTML. Отправь форму и найди POST. Проверь наличие Set-Cookie в успешном ответе, затем Cookie в следующем запросе. Не копируй значения билетов и паролей в заметки.

Для комнаты найди Blazor-соединение, обычно к endpoint с префиксом _blazor. В WebSocket frames увидишь сообщения, но не обязательно быть удобным JSON: протокол framework может быть бинарным. Для понимания достаточно сопоставить время клика, серверного breakpoint и обновления UI.

Поставь breakpoint в SendAsync, ChatService.SendMessageAsync и метод репозитория. Пройди путь. Между компонентом и сервисом не появится запрос api/messages в DevTools, потому что это прямой вызов в серверном процессе.

Для lifecycle логируй RoomId и Guid экземпляра в OnInitializedAsync, OnParametersSetAsync и Dispose. Для DI добавь Guid в диагностический scoped-сервис. Сравни две вкладки и два обращения внутри одной. Это покажет границы объектов лучше, чем заучивание таблицы lifetime.

Симптом	Первый вопрос
Headers are read-only	В каком ответе пытаемся изменить заголовки?
Unable to resolve service	Какая зависимость не зарегистрирована?
Second operation on context	Кто разделяет один DbContext?
Сообщение в БД, но не в UI	Какой путь уведомления не прошёл?
После перехода старая история	Не завершилась ли устаревшая загрузка?

После эксперимента закрой исходники и объясни наблюдение: где жил объект, что пришло по сети, что изменилось и почему. Если ответ только «Blazor так делает», выбери следующий более узкий эксперимент.

Вернуться по памяти: Как доказать прямой вызов ChatService, используя одновременно DevTools и debugger?

Для проверки деталей: [23], [10], [19]. Ссылки на с. 39-40.

Практика 1. Три режима одной кнопки

Сначала предсказать, потом запускать.

Работай в отдельной учебной ветке или маленьком проекте. Не меняй рабочую аутентификацию ради эксперимента. Перед каждым запуском запиши ожидаемый путь: будет ли HTTP-запрос, где исполнится C# и что останется после обновления страницы.

А. Статическая страница

Сделай страницу со счётчиком и `button @onclick`. Не назначай ей интерактивный режим и убедись, что `Routes` тоже статический. Открой страницу и нажми кнопку. Предскажи, должен ли сработать breakpoint в C#-обработчике. Объясни, почему HTML-кнопка всё равно реагирует на мышшь визуально.

Б. Interactive Server

Добавь странице `@rendermode InteractiveServer`. Снова открой страницу, нажми кнопку и посмотри `Network`. Сработал ли обработчик? Появился ли новый запрос документа на каждый клик? Затем обнови страницу полностью: должен ли счётчик сохраниться без специального хранилища?

В. Статическая форма

Сделай отдельную static SSR страницу с `EditForm`, `method="post"`, `FormName` и `SupplyParameterFromForm`. Отправь строку и выведи её после обработки. Сравни: когда значение появилось в серверной модели - после набора каждой буквы или только после POST?

Г. Два экземпляра

Добавь `Guid` экземпляра и лог `OnInitializedAsync`. Включи `prerender`. Открой страницу полностью и сопоставь сообщения. Затем открой вторую вкладку. Попробуй объяснить количество экземпляров без слов «глюк framework». Не предполагай, что любой переход обязательно создаёт новый объект.

Результат упражнения: четыре короткие записи «действие - транспорт - место выполнения - время жизни». Скриншот кнопки без объяснения не показывает усвоение механизма.

Вернуться по памяти: Через день: восстанови различие static SSR и Interactive Server по памяти, не открывая код.

Практика 2. Наш вход и наш чат

Собрать цельную модель из наблюдений.

1. Вход

Открой новую приватную сессию браузера. Посети /login. Введи неправильный пароль и объясни, почему валидация Required прошла, а вход нет. Затем войди правильно. Найди POST, успешный ответ и последующий запрос главной страницы. Отметь направление Cookie и Set-Cookie.

2. Прямой доступ к Application

Поставь breakpoint в метод загрузки комнат компонента и в ChatService.GetRoomsAsync. Пройди вызов. Сравни с WASM-этапом: где раньше был HTTP-запрос api/rooms и где он теперь отсутствует? Укажи, какая сеть всё ещё остаётся между браузером и сервером.

3. Два пользователя

Нарисуй два компонента Room и общий notifier. Добавь цепочку сохранения. Отметь, кто вызывает ChatService, кто публикует сообщение и кто иницирует рендеринг второго UI. Для приватного сообщения зачеркни недопустимые направления доставки.

4. Две вкладки одного пользователя

Предскажи, одинаковы ли их cookie, circuit, черновик и scoped ChatService. Объясни, почему presence должна считать участия. Затем предскажи последствия закрытия одной вкладки, обрыва сети и перезапуска серверного процесса.

5. Ревизия lifetime

Открой AddInfrastructure и репозитории. Определи, кто владеет DbContext и когда он освобождается. Если компонент держит scoped ChatService всю жизнь circuit, предложи границу операции. Перед изменением регистраций перечисли все зависимости UserManager, Identity stores и репозиториях, которых изменение может затронуть.

После каждой цельной работающей правки делай коммит. Название описывает итоговое поведение, например Add static SSR sign-in или Use an operation scope for chat queries. Диагностический лог сам по себе не требует отдельного этапа архитектуры.

Вернуться по памяти: Итог: объясни за пять минут открытие страницы, вход и доставку нового сообщения второму пользователю.

Самопроверка: опорные ответы

Открой эту страницу после собственной попытки.

Тег и транспорт. HTML button может участвовать в интерактивности через @onclick. EditForm может отправлять HTTP POST в static SSR. Нужно узнать режим и способ обработки действия, а не только название тега.

Cookie. Set-Cookie идёт от сервера браузеру в HTTP-ответе. Cookie идёт от браузера серверу в подходящих запросах. SignInManager готовит cookie через authentication handler; он не пишет непосредственно в память браузера.

Интерактивный вход. Обработчик клика работает на сервере, но не имеет нового обычного ответа страницы для выдачи cookie. Серверный HttpClient тоже не решает это автоматически: его ответ получает он сам.

Circuit. Это серверный UI-сеанс, а не cookie, отдельный поток или только сокет. Две вкладки обычно имеют разные circuits и поля компонентов, хотя используют общую браузерную cookie.

Рендеринг. Изменение C#-поля меняет состояние сервера. Renderer вычисляет вывод и изменения, клиент применяет их к DOM. StateHasChanged не читает БД автоматически и не гарантирует немедленное завершение передачи.

Scoped и DbContext. В HTTP scope короткий, в circuit долгий. Поэтому перенос scoped сервиса из API в компонент меняет срок использования context. AsNoTracking не делает context безопасным для перекрывающихся операций.

Доставка сообщения. БД обеспечивает хранение. Notifier сообщает подписчикам. Компонент проверяет принадлежность события и инициирует UI-обновление. Один процесс не уведомляет другой через свой singleton автоматически.

Prerender. Сначала создаётся предварительный вывод в рамках HTTP, затем интерактивный экземпляр. Двойной лог инициализации может быть нормальным. Запись данных в инициализации требует особой осторожности.

Выбор архитектуры. Server проще для прямых серверных сценариев. WASM переносит локальную UI-логику в браузер. Большое количество функций не является достаточным основанием выбора. Требования к сети, задержке, аудитории и локальному состоянию важнее.

Если один ответ пока нельзя объяснить своим примером, вернись к соответствующему разделу и проведи один эксперимент. Не превращай страницу ответов в текст для заучивания.

Источники и дальнейшее чтение

Официальная документация и исходники framework.

Номера внизу разделов отсылают к этому списку. Используйте документацию для деталей API после восстановления общей модели. При чтении Microsoft Learn выбери .NET 10 в селекторе версии; старые страницы Blazor Server и современные Blazor Web App могут описывать разные настройки.

[1] Режимы рендеринга

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/render-modes?view=aspnetcore-10.0>

[2] Модели размещения Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/hosting-models?view=aspnetcore-10.0>

[3] HTTP middleware и порядок обработки

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0>

[4] Обработка событий компонентов

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/event-handling?view=aspnetcore-10.0>

[5] Рендеринг компонентов

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/rendering?view=aspnetcore-10.0>

[6] Формы и antiforgery

<https://learn.microsoft.com/en-us/aspnet/core/blazor/forms/?view=aspnetcore-10.0>

[7] Kestrel

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/servers/kestrel?view=aspnetcore-10.0>

[8] Reverse proxy и forwarded headers

<https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/proxy-load-balancer?view=aspnetcore-10.0>

[9] Dependency injection в ASP.NET Core

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-10.0>

[10] Dependency injection в Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/dependency-injection?view=aspnetcore-10.0>

[11] Lifecycle компонентов

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/lifecycle?view=aspnetcore-10.0>

[12] Binding статических серверных форм

<https://learn.microsoft.com/en-us/aspnet/core/blazor/forms/binding?view=aspnetcore-10.0>

Источники: состояние и безопасность

Официальная документация и исходники framework.

Для cookie-входа сравни пример с исходником шаблона. В репозитории выбери ветку `release/10.0`: `main` может относиться к следующему выпуску. Шаблон содержит дополнительные сценарии: блокировку, двухфакторный вход, внешние providers и другие детали. Они не нужны для понимания базового HTTP-обмена, но важны при развитии приложения.

[13] Аутентификация и авторизация Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/security/?view=aspnetcore-10.0>

[14] SignalR-инфраструктура Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/signalr/?view=aspnetcore-10.0>

[15] Prerendering

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/prerender/?view=aspnetcore-10.0>

[16] Развёртывание server-side Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/host-and-deploy/server/?view=aspnetcore-10.0>

[17] Освобождение компонентов

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/component-disposal/?view=aspnetcore-10.0>

[18] Synchronization context

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/synchronization-context/?view=aspnetcore-10.0>

[19] EF Core в server-side Blazor

<https://learn.microsoft.com/en-us/aspnet/core/blazor/blazor-ef-core/?view=aspnetcore-10.0>

[20] Прикладные SignalR Hubs

<https://learn.microsoft.com/en-us/aspnet/core/signalr/hubs/?view=aspnetcore-10.0>

[21] Cookie authentication

<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/cookie/?view=aspnetcore-10.0>

[22] Data Protection

<https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/introduction/?view=aspnetcore-10.0>

[23] HttpContext и изменение заголовков

<https://learn.microsoft.com/en-us/aspnet/core/blazor/components/httpcontext/?view=aspnetcore-10.0>

[24] Навигация и маршрутизация

<https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/routing/?view=aspnetcore-10.0>

[25] Исходник login: выбери ветку `release/10.0`

<https://github.com/dotnet/aspnetcore/blob/main/src/ProjectTemplates/Web.ProjectTemplates/content/BlazorWeb-CSharp/BlazorWebCSharp.1/Components/Account/Pages/Login.razor>